

Day 5 : Material Design Widgets

1. Introduction to Material Design

Material Design is a design system developed by Google that provides guidelines for creating attractive, consistent, and user-friendly applications. Flutter follows Material Design principles through its Material Widgets library. These widgets help developers create applications that look modern and work smoothly across Android, iOS, Web, and Desktop platforms.

Material Design focuses on:

- Consistent user experience
- Attractive visual appearance
- Easy navigation
- Better accessibility
- Responsive layouts
- Interactive animations

2. MaterialApp Widget

The MaterialApp widget is the root widget of most Flutter applications. It provides Material Design styling and functionality to the entire application.

Responsibilities of MaterialApp

- Sets up application theme.
- Defines routes for navigation.
- Controls app title.
- Manages navigation between screens.
- Applies Material Design styling.

```
MaterialApp(  
  title: 'My App',  
  home: HomeScreen(),  
);
```

3. Scaffold Widget

Scaffold provides the basic structure of a screen.

Components of Scaffold

- AppBar
- Body
- Floating Action Button

- Drawer
- Bottom Navigation Bar
- SnackBar support

```
Scaffold(  
  appBar: AppBar(  
    title: Text("Dashboard"),  
  ),  
  body: Center(  
    child: Text("Welcome"),  
  ),  
);
```

4. Material Design Buttons

ElevatedButton

ElevatedButton appears raised above the surface with shadow effects.

Characteristics

- Has elevation (shadow).
- Attracts user attention.
- Suitable for primary actions.

Common Uses

- Login
- Submit Form
- Save Data
- Confirm Action

```
ElevatedButton(  
  onPressed: () {},  
  child: Text("Login"),  
)
```

TextButton

TextButton is a flat button without elevation.

Characteristics

- No background color by default.
- Simple appearance.

Common Uses

- Cancel
- Skip
- Learn More
- Secondary Actions

```
TextButton(  
  onPressed: () {},  
  child: Text("Cancel"),  
)
```

OutlinedButton

OutlinedButton displays a border around the button.

Characteristics

- Visible border.
- Medium emphasis action.

Uses

- Register
- View Details
- Alternative Action

```
OutlinedButton(  
  onPressed: () {},  
  child: Text("Register"),  
)
```

IconButton

IconButton displays an icon that users can tap.

Uses

- Search
- Delete
- Settings
- Notifications

```
IconButton(  
  icon: Icon(Icons.search),  
  onPressed: () {},  
)
```

FloatingActionButton (FAB)

Floating Action Button is a circular button usually placed at the bottom-right corner.

Characteristics

- Represents primary action.
- Always visible.

Uses

- Add Contact
- Create Note
- Add Product

```
FloatingActionButton(  
  onPressed: () {},  
  child: Icon(Icons.add),  
)
```

5. Card Widget

Cards group related information into a visually appealing container.

Features

- Rounded corners
- Shadow effect
- Organized information display

```
Card(  
  elevation: 5,  
  child: Padding(  
    padding: EdgeInsets.all(10),  
    child: Text("Student Information"),  
  ),  
)
```

6. ListTile Widget

ListTile is used for displaying structured information in lists.

Components

Leading

Displays content before the title.

```
ListTile(  
  leading: Icon(Icons.person),  
  title: Text("Sneha"),  
  subtitle: Text("Developer"),  
  trailing: Icon(Icons.arrow_forward),  
)
```

7. Divider Widget

Divider creates a horizontal line between UI elements.

8. Dialogs in Flutter

Dialogs display important information or request user input.

1. AlertDialog

Shows warning messages or confirmation requests

```
AlertDialog(  
  title: Text("Delete"),  
  content: Text("Are you sure?"),  
);
```

2. SimpleDialog

Provides multiple choices to users.

```
SimpleDialog(  
  title: Text("Choose Option"),  
);
```

3. showDialog()

Used to display dialogs.

```
showDialog(  
  context: context,  
  builder: (context) => AlertDialog(),  
);
```

4. Custom Dialog

Developers can design personalized dialogs according to application requirements.

Uses:

- User profile popup
- Custom forms
- Payment confirmations

5. SnackBar

SnackBar shows temporary messages at the bottom of the screen.

```
ScaffoldMessenger.of(context).showSnackBar(  
  SnackBar(  
    content: Text("Data Saved"),  
  ),  
);
```

6. SnackBar Actions

Allows users to undo or retry actions.

```
SnackBar(  
  content: Text("Item Deleted"),  
  action: SnackBarAction(  
    label: "UNDO",  
    onPressed: () {},  
  ),  
);
```

9 . Forms in Flutter

Forms collect user information and validate input

1. TextField

Basic input widget.

```
TextField(  
  decoration: InputDecoration(  
    labelText: "Name",  
  ),  
);
```

2. TextFormField

Provides built-in validation support.

```
TextFormField(  
  validator: (value) {  
    if(value!.isEmpty){  
      return "Enter Name";  
    }  
    return null;  
  },  
);
```

3. Form Widget

Groups multiple input fields together.

```
Form(  
  child: Column(  
    children: [],  
  ),  
);
```

10. Navigation in Flutter

Navigation helps move between screens

1. Navigator.push()

Opens a new screen.

```
Navigator.push(  
  context,  
  MaterialPageRoute(  
    builder: (context) => SecondScreen(),  
  ),  
);
```

2. Navigator.pop()

Returns to the previous screen.

Navigator.pop(context);

1. Demo

